Clash of Java

Memoria del Proyecto

Alumno/a: Mario Cabello Tirso

Grupo: 1º DAW

Curso Académico: 2026/2027

Repositorio GitHub: https://github.com/mctirso1502/ClashOfJava

---

1. Descripción del Proyecto

1.1. Objetivo General

El proyecto **"Clash of Java"** es un minijuego de gestión de recursos y probabilidad que funciona íntegramente en la consola de comandos usando Java. Está inspirado en la parte de gestión de aldeas de juegos populares. El objetivo de este trabajo es poner en práctica los conocimientos de programación estructurada que hemos dado en clase: lectura de datos por teclado con `Scanner`, bucles (especialmente `do-while` para los menús), condicionales complejos y el uso de arrays para almacenar información estructurada.

1.2. Reglas y Mecánicas del Juego

El juego no tiene un final estricto donde ganes o pierdas para siempre, sino que es un juego de progresión continua. Las reglas y mecánicas principales son las siguientes:

- **Recursos iniciales:** El jugador empieza con una cantidad generosa de recursos (90.000 de Oro y 90.000 de Elixir) para poder empezar a jugar y mejorar su aldea rápido.
- **Edificios:** Hay dos edificios principales, el Ayuntamiento y el Cañón. Ambos empiezan a nivel 1 y tienen un nivel máximo fijado en 10 (controlado por la constante `MAXNIVEL`).
- **Economía de Mejoras:** Mejorar el Ayuntamiento cuesta Oro, y mejorar el Cañón cuesta Elixir. El coste no es fijo, sino que se va encareciendo a medida que subes de nivel según una fórmula matemática.
- **Recolección manual:** Si te quedas sin recursos, puedes "farmear" (recolectar) Oro o Elixir de forma manual entrando en las opciones de recolección y pulsando la tecla "Intro" repetidamente. Cada pulsación otorga 2.500 unidades del recurso.
- **Batallas por probabilidad:** Las batallas no son combates de mover fichas, sino tiradas de suerte. Cuestan 10.000 de Elixir para participar. La probabilidad de ganar depende directamente del nivel de tu Cañón (a mejor cañón, más posibilidades de victoria). Si ganas, te llevas un botín enorme; si pierdes, pierdes el elixir apostado.

1.3. Desarrollo de la Partida (El Menú Principal)

Nada más ejecutar el código, el programa te da la bienvenida y entra en un bucle `do-while` infinito que muestra un menú con 7 opciones. El jugador escribe un número para elegir qué hacer:

1. **Ver recursos:** Te muestra cuánto Oro y Elixir tienes ahorrado.
2. **Ver niveles:** Te dice a qué nivel tienes actualmente el Ayuntamiento y el Cañón.
3. **Mejoras:** Abre un submenú para gastar tus recursos en subir de nivel los edificios.
4. **Recolectar Oro:** Entras en un modo donde pulsando "Intro" ganas Oro gratis.
5. **Recolectar Elixir:** Igual que el anterior, pero para Elixir.
6. **Batalla:** Te permite apostar Elixir para intentar ganar el premio gordo.
7. **Salir:** Rompe el bucle principal y cierra el programa.

Después de cada acción, el programa usa `sc.nextLine();` de forma estratégica para hacer una pausa, obligando al usuario a pulsar Intro antes de volver a imprimir el menú principal. Esto evita que el texto pase volando por la consola y no se pueda leer.

2. Funcionalidades Implementadas y Diferencias

2.1. Funcionalidades Detalladas

Para hacer que la progresión del juego sea interesante, he programado las siguientes funcionalidades matemáticas y lógicas:

- **Cálculo dinámico de costes:** No he puesto un precio fijo para las mejoras. He usado la fórmula `(costeBase + (nivelActual * 30000))`. Así, la primera mejora cuesta 90.000 (60k + 30k), pero para subir al nivel 5 te costará mucho más. Esto obliga al jugador a recolectar más dinero conforme avanza el juego.
- **Control de máximos:** He implementado una barrera de nivel usando una condición `if (ayuntamiento >= MAXNIVEL)`. Si el jugador intenta mejorar un edificio de nivel 10, el juego lo bloquea y le avisa.
- **Confirmación de seguridad:** En la tienda de mejoras, antes de restarte el dinero, el programa entra en un bucle de confirmación que te pregunta "¿Seguro que quieres continuar? (S/N)". Hasta que no tecleas una 'S' o una 'N' mayúscula, no te deja salir de esa pantalla.
- **Detección de tecla "Intro" vacía:** Para los menús de recolección, logré implementar un sistema que detecta si el usuario le ha dado a "Enter" sin escribir nada usando la función `pulsador.isEmpty()` de los Strings.
- **Sistema RNG (Random Number Generator) equilibrado:** En la batalla, la tirada se hace con `(int)(Math.random() * 101)`, generando un número del 0 al 100. La gracia es que la probabilidad de ganar base es del 30%, pero por cada nivel del cañón se suma un 5% extra. `probabilidadGanar = 30 + (canion * 5);`. A nivel 10, tienes un 80% de probabilidad de ganar.

2.2. Diferencias respecto a la propuesta inicial

Al principio del proyecto pensé en hacer un juego mucho más complejo, pero tuve que adaptarlo al tiempo disponible y a los conocimientos actuales:

- **Menos recursos:** Iba a poner también Gemas y Madera, pero para simplificar el uso de los arrays y no alargar los menús infinitamente, lo dejé solo en Oro y Elixir.
- **Sin mapas:** Originalmente, "Clash of Java" iba a tener un tablero donde poner las defensas, pero dibujar mapas en la consola con matrices era demasiado propenso a errores de visualización. En su lugar, enfoqué el proyecto al 100% en la gestión matemática de los recursos, asegurándome de que los cálculos de economía funcionaran perfectamente.

3. Diseño del Código

3.1. Variables y Constantes

El diseño de variables es muy limpio y se declaran todas al principio del `main`.

- **Constantes:** He creado `final int MAXNIVEL = 10;`. Usar esta constante es una buena práctica porque si mañana quiero que el juego dure hasta el nivel 20, solo tengo que cambiar ese "10" por un "20" en la primera línea, y todos los menús de mejora se actualizarán automáticamente sin tener que reescribir el código.
- **Variables acumuladoras:** `ayuntamiento` y `canion`, que empiezan en 1 y se van incrementando con `++`.
- **Variables de uso temporal:** `opcion`, `opcionMejora`, `pulsador` (para recoger textos del teclado) y `tirada` (para guardar el número aleatorio de la batalla).

3.2. Estructuras de Control de Flujo

El esqueleto de mi programa son las estructuras de control. Todo el juego está sostenido por un enorme `do-while (opcion != 7)`. Dentro de este bucle principal, se encuentra un `switch (opcion)` gigante que dirige el flujo hacia cada uno de los 7 sub-programas.

Lo más complejo del diseño fue **anidar estructuras**. Por ejemplo, la Opción 3 (Mejoras) tiene su propio menú. Esto significa que dentro del `switch` principal hay otro bucle `do-while (opcionMejora != 3)`, y dentro de ese bucle, hay *otro* `switch` para elegir si mejoras el Ayuntamiento o el Cañón. Y por si fuera poco, dentro de cada caso hay *otro* `do-while` para validar si el usuario pulsa 'S' o 'N'. Esta arquitectura de bucles dentro de bucles me ha permitido hacer menús muy seguros donde el jugador no se puede quedar atascado.

3.3. Explicación detallada del uso de los Arrays

El apartado más importante de los requisitos del proyecto era la implementación de Arrays. Para el motor económico del juego, decidí no usar variables sueltas (como `int oro; int elixir;`), sino agruparlas en un array unidimensional numérico.

En el código se declara e inicializa así: `int [] recursos = {90000, 90000};`

¿Por qué usar un Array para esto? Al agrupar la economía en un array llamado `recursos`, he centralizado los "dineros" del jugador. La estructura lógica que he diseñado es:

- **Posición 0 (`recursos[0]`):** Almacena exclusivamente el **Oro**. El oro se usa principalmente para mejorar el Ayuntamiento.
- **Posición 1 (`recursos[1]`):** Almacena exclusivamente el **Elixir**. El elixir se usa para mejorar el Cañón y para pagar el ticket de entrada a las Batallas.

Operaciones con el Array durante la ejecución:

1. **Lectura:** Cuando el jugador elige la Opción 1 del menú ("Ver recursos"), el programa accede a los índices del array y los concatena en un mensaje de texto.

```
System.out.println("Oro: " + recursos[0]);
System.out.println("Elixir: " + recursos[1]);
```

2. **Asignación por sumas (Recolección y Premios):** Cuando el jugador está en la sección de "Recolectar Oro", cada vez que pulsa Intro, el programa accede a la posición 0 del array, coge el valor que hay en ese momento, le suma 2500 y vuelve a machacar la variable con el nuevo resultado. Esto se hace con la instrucción: `recursos[0] = recursos[0] + 2500;`. Lo mismo ocurre cuando se gana una batalla, pero sumando 90.000 de golpe a ambos índices simultáneamente.

3. **Comprobación y Restas (Compras):** La parte más crítica del uso del array es verificar si hay fondos suficientes antes de hacer una compra. En la línea 48 del código, al intentar mejorar el ayuntamiento, el programa calcula el coste total y luego lo compara con el array:

```
if ((costeAyuntamiento + (ayuntamiento * 30000)) > recursos[0]) { ... }
```

Si esa condición es falsa (es decir, sí tenemos dinero suficiente), el programa procede a cobrar restando el valor del índice 0:

```
recursos[0] = recursos[0] - (costeAyuntamiento + (ayuntamiento * 30000));
```

Aunque de momento es un array de solo 2 posiciones, usar esta estructura me permite que en un futuro (en la segunda evaluación), si decido añadir 10 recursos diferentes (gemas, madera, piedra, hierro, comida...), solo tendré que hacer el array más grande `int[] recursos = new int[10]` y podré recorrerlo con un simple bucle `for` para imprimirlos todos, en lugar de tener que hacer 10 `System.out.println` sueltos. Es una forma de programar pensando en el futuro.

4. Pruebas Realizadas y Resultados

He realizado sesiones de prueba (testing) de todas las opciones del menú para asegurarme de que el programa no "crashea" y que las matemáticas funcionan bien.

4.1. Casos Normales

Prueba	Acción del usuario	Resultado esperado	Resultado real
Mejorar Ayuntamiento	Voy a la Opción 3, luego 1. Confirmo con 'S'.	Me resta 90.000 de Oro (estando a nivel 1) y el Ayuntamiento sube a nivel 2.	**ÉXITO.** El saldo de Oro bajó a 0 y el nivel subió a 2.
Recolectar recursos	Entro a la Opción 4 (Recolectar Oro) y pulso "Intro" 3 veces. Luego pulso "1".	Me suma 2500 x 3 = 7500 de Oro y salgo al menú.	**ÉXITO.** Se sumó perfectamente usando el `pulsador.isEmpty()`.
Batalla con éxito	Entro a Batalla (Opción 6), pago 10.000. El número aleatorio sale bajo.	Aparece "¡HAS GANADO!" y se suman 90.000 a ambos recursos.	**ÉXITO.** El botín se sumó correctamente al array.

4.2. Casos Incorrectos y Filtros

Prueba	Acción del usuario	Resultado esperado	Resultado real
Confirmación equivocada	En la compra, cuando pregunta (S/N), escribo "H" o "123".	El programa avisa de que no es válido y me vuelve a preguntar.	**ÉXITO.** El `while (continuar != 'S' && continuar != 'N')` me atrapa en un bucle seguro.
Comprar sin dinero	Intento mejorar el Cañón (cuesta 90k) pero solo tengo 500 de Elixir en `recursos[1]`.	Me avisa de que cuesta X y solo tengo Y, bloqueando la compra.	**ÉXITO.** El `if` frena la operación y me devuelve al menú de mejoras.

4.3. Casos Límite y Extremos

Prueba	Acción del usuario	Resultado esperado	Resultado real
Nivel Máximo (10)	Con trucos de dinero, subo el Ayuntamiento hasta el nivel 10. Luego intento subirlo al 11.	El juego me muestra el aviso "Has llegado al nivel máximo!" y no me cobra dinero.	**ÉXITO.** La constante `MAXNIVEL` funciona como tope.

Prueba	Acción del usuario	Resultado esperado	Resultado real
Ir a batalla pobre	Gasto todo el elixir. Con 0 de elixir intento ir a la opción 6 (Batalla).	El programa me dice que necesito 10.000 y me echa al menú principal.	**ÉXITO.** El filtro inicial `if (recursos[1] < 10000)` funciona bien.

5. Dificultades, Soluciones y Posibles Ampliaciones

5.1. Dificultades Técnicas Encontradas

Durante el desarrollo del código, me enfrenté a dos grandes problemas técnicos que me llevaron horas entender, pero que conseguí solucionar:

1. El Bug del Scanner (Saltos de línea fantasmas):

El problema: En la Opción 3 (Mejoras), usé `opcionMejora = sc.nextInt();` para leer el número del menú. El problema es que `nextInt()` lee el número que has tecleado, pero deja el carácter "Enter" flotando en la memoria del programa (el buffer). Cuando más adelante el programa volvía al menú principal e intentaba leer texto o hacer una pausa, se "tragaba" ese Enter fantasma, se volvía loco y los menús se imprimían dos veces seguidas sin darte tiempo a leer nada.

La solución: Tuve que estudiar cómo funciona la memoria del Scanner. La solución fue meter "limpiaparabrisas" en el código. Justo debajo de cada `sc.nextInt();`, escribí un `sc.nextLine();` suelto (como se ve en la línea 44 del código: `sc.nextLine(); // Limpiar buffer`). Esta línea absorbe el Enter residual y deja la consola limpia para la siguiente pregunta. Además, en el menú principal usé `Integer.parseInt(sc.nextLine());`, que es una forma mucho más limpia y profesional de pedir números sin generar basura en el buffer.

2. Recolección pulsando solo "Intro":

El problema: Yo quería que farmear recursos fuera cómodo. No quería obligar al jugador a teclear "1", luego "Enter", luego "1", luego "Enter". Quería que simplemente aporreando la tecla Enter, el dinero subiera.

La solución: Usé la lectura de strings completos `pulsador = sc.nextLine();`. Si el jugador pulsa Enter sin escribir ninguna letra, la variable `pulsador` se queda vacía `""`. Gracias a la función integrada `pulsador.isEmpty()`, el programa detecta ese vacío y ejecuta el bloque de sumar dinero `recursos[0] = recursos[0] + 2500;`. Si el jugador escribe un "1", se sale. Fue la mecánica más divertida de programar.

5.2. Posibles Ampliaciones para la 2ª Evaluación

El juego está muy estable, pero como es en consola, se puede hacer mucho más grande. Para la siguiente evaluación, cuando sepamos más programación, me gustaría añadirle lo siguiente:

- **Guardado en ficheros:** Ahora mismo, si eliges la opción 7 (Salir), pierdes todo el Oro y el nivel de los edificios. En el futuro, haré que antes de cerrarse, el programa escriba un archivo `partida.txt` en el disco duro guardando el array de recursos y los niveles, y que al arrancar, el programa lea ese archivo para que puedas continuar tu partida otro día.
- **Menú de Tienda Complejo (Objetos):** Me gustaría crear más edificios (Muros, Minas, Cuarteles). En vez de tener variables sueltas como `ayuntamiento` y `canion`, usaré Programación Orientada a Objetos para crear la clase `Edificio`, y guardaré todos los edificios del jugador en un gran Array de Objetos.
- **Mejora en la captura de errores (Excepciones):** Ahora mismo, si el menú principal te pide que escribas un número (Opción del 1 al 7) y por error pulsas una letra, el programa da un error rojo (`NumberFormatException`) y se detiene. Me gustaría añadir un bloque `try-catch` para atrapar ese error y que simplemente te diga "Por favor, introduce un número válido", sin cerrarle el juego al usuario.